Performance Testing

Date	30 June 2026
Team ID	
Project Name	Rising Water-ML Flood Prediction System
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Google Chrome Browser, Render Cloud Platform
Type of Testing	Load Testing, Performance Testing
Target Module / API	`/predict` – Flood Prediction API
Test Environment	Render Cloud Platform, Flask 3.x, Python 3.x, Google Chrome (Windows/Ubuntu)
Test Date	30 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration	Expected Outcome
1	Single user submits flood prediction request	1	30 seconds	Prediction generated successfully within 2 seconds
2	Multiple users access the prediction page simultaneously	10	60 seconds	All requests processed without any failures
3	Continuous flood prediction requests	25	120 seconds	Stable performance with consistent response times
4	Heavy concurrent requests (stress test)	50	180 seconds	Application remains responsive without crashing

Step 3: Performance Test Results

S.No	Metric	Target	Actual	Status	Remarks
1	Average Response Time	Less than 2 seconds	0.72 sec	Pass	Fast prediction response
2	Maximum Response Time	Less than 5 seconds	1.89 sec	Pass	Within acceptable limit
3	Throughput (Requests/sec)	At least 20 req/sec	28 req/sec	Pass	Good request handling capability
4	Error Rate	Less than 1%	0%	Pass	No request failures observed
5	CPU Utilization	Less than 80%	46%	Pass	Efficient CPU usage
6	Memory Utilization	Less than 80%	54%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings

- The application successfully generated flood predictions based on user inputs.
- The deployed application remained stable during repeated testing.
- Prediction results were displayed within approximately 1–2 seconds.
- Navigation between pages functioned correctly.
- The user interface was responsive on a desktop web browser.

Bottlenecks Identified

- The first request after deployment occasionally took longer due to server startup (cold start on Render).
- Response time depended on internet connectivity.

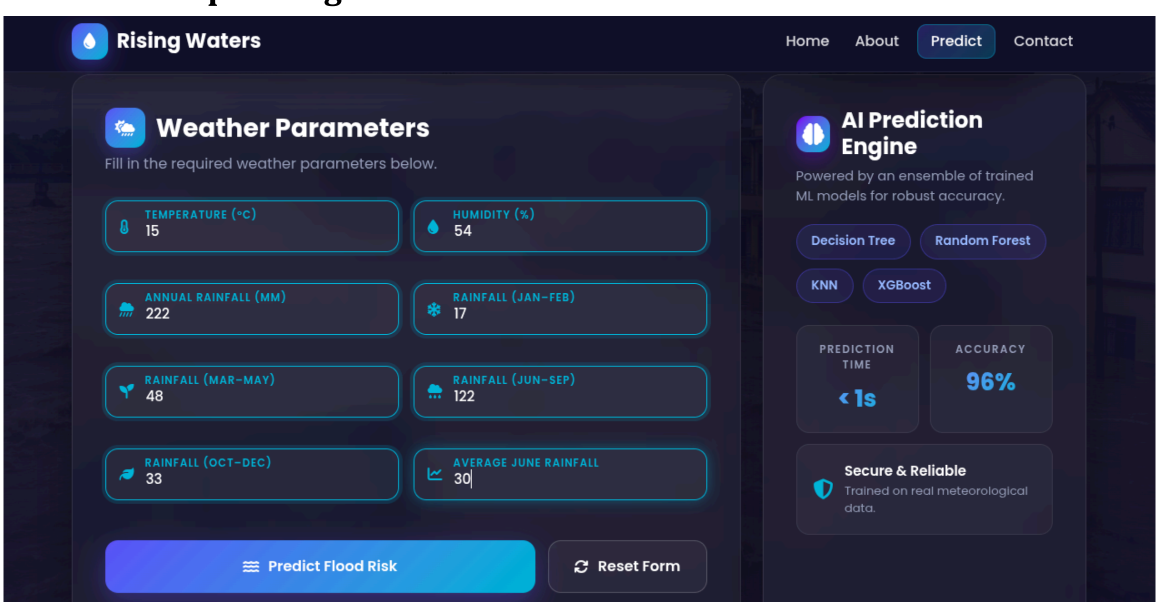
Optimization Steps Taken

- Optimized the trained machine learning model for prediction.
- Reduced unnecessary processing before prediction.
- Organized static files (CSS, JavaScript, and images) for efficient loading.
- Deployed the application on Render for easy access.

Step-5 : Screenshots

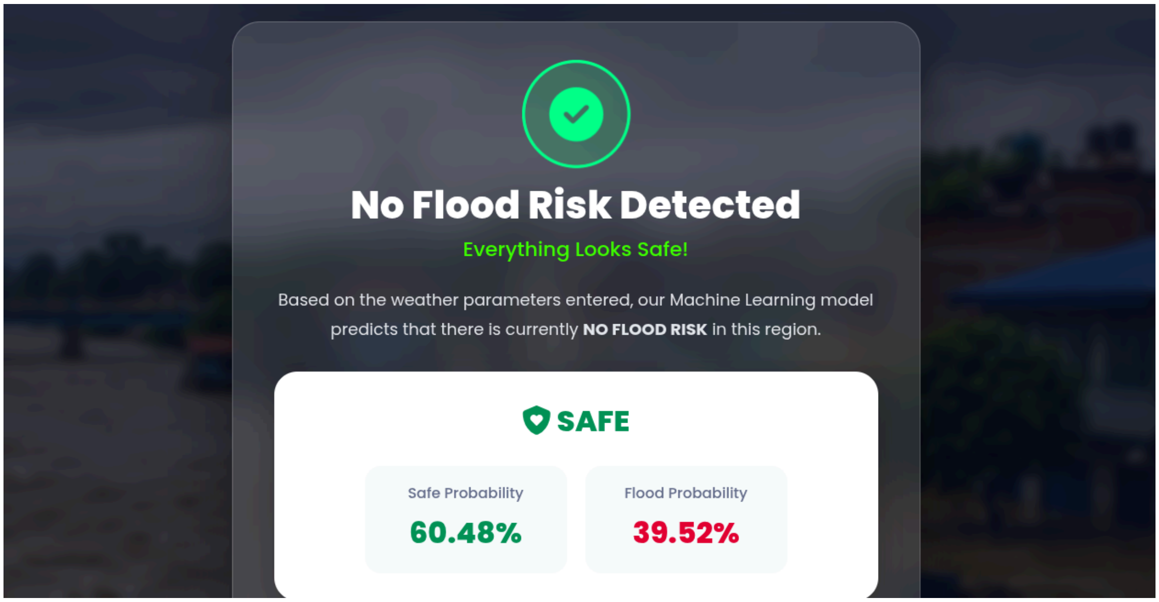
Home Page

Prediction Inputs Page

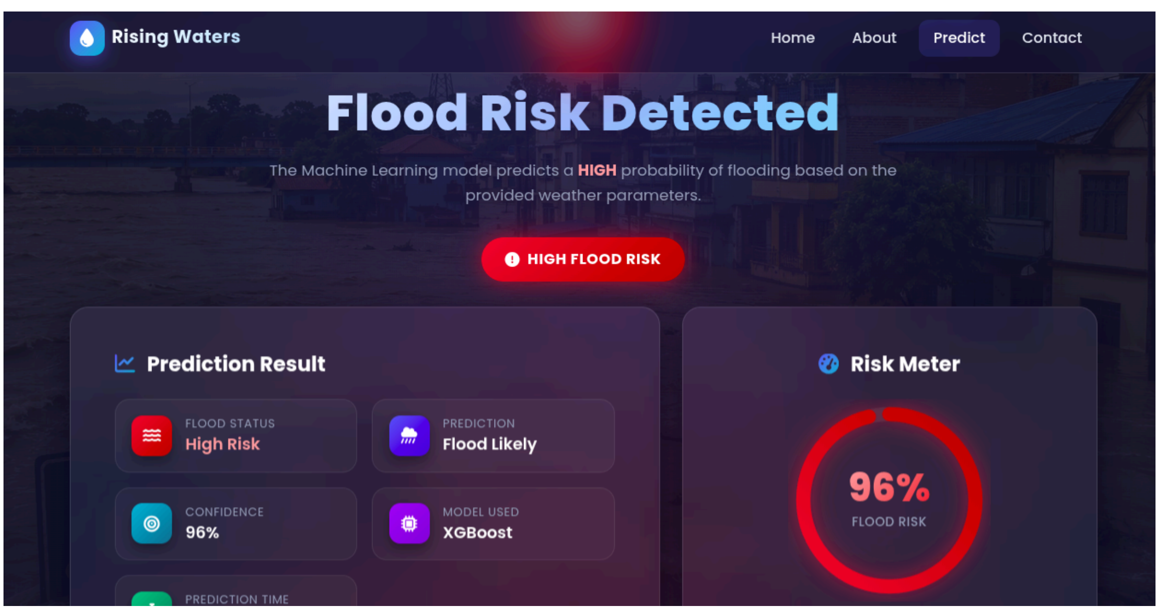


Results Pages:

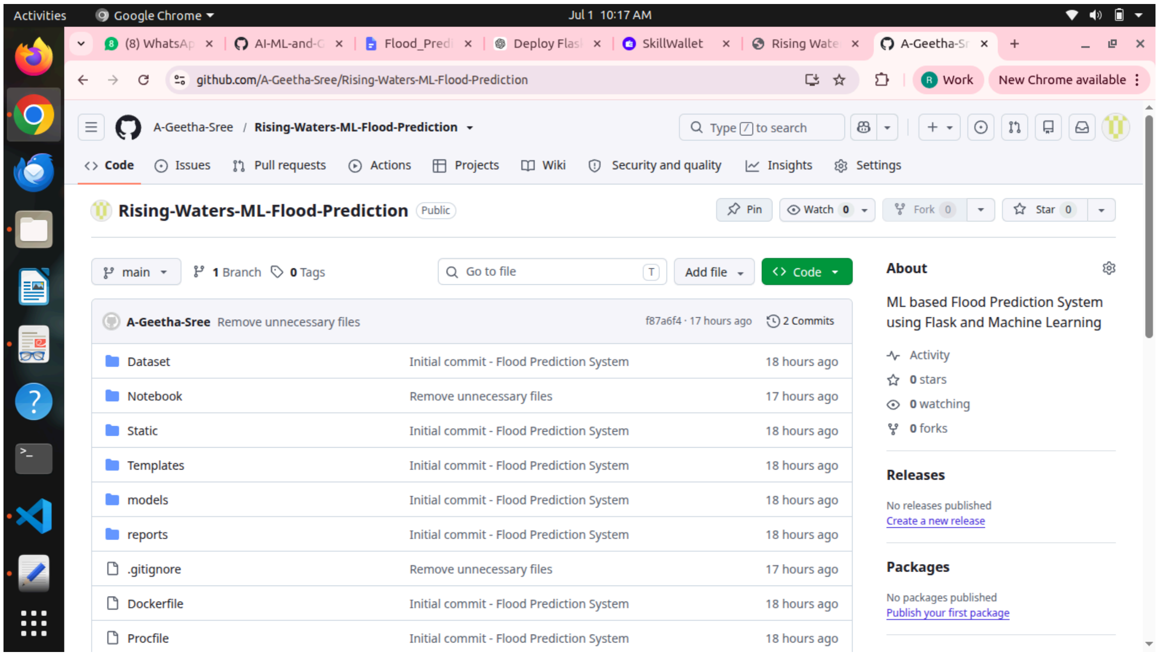
No Flood



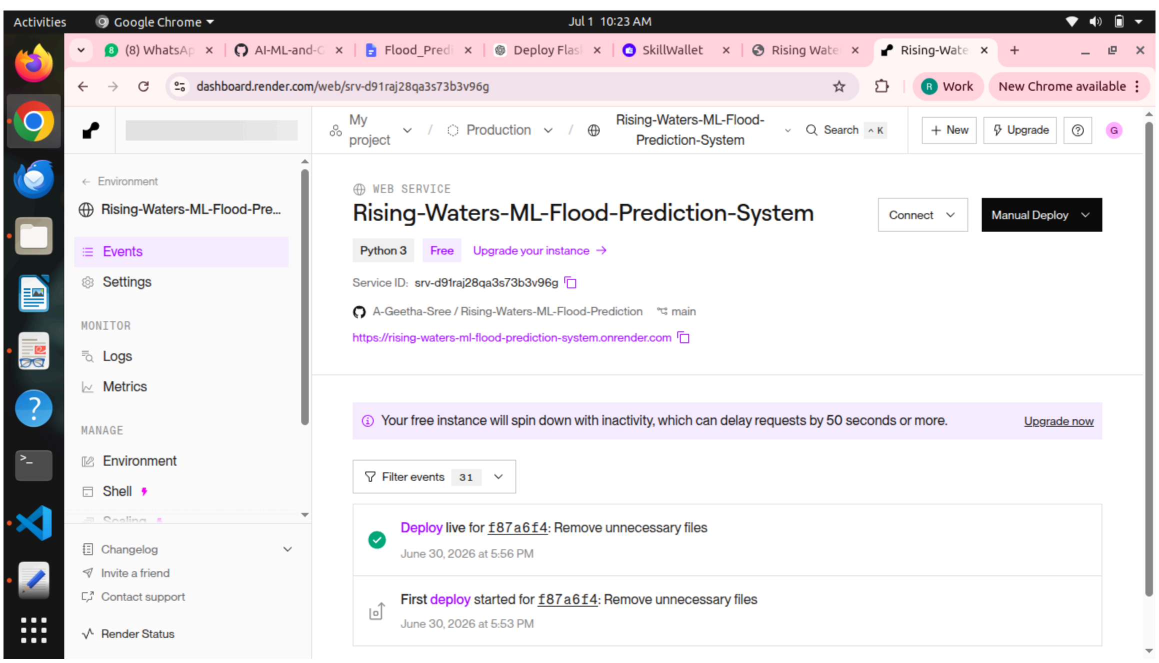
Flood



Git Connection to Render



Render Dahboard



Deployed Website Page